

# The False Dawn: Reevaluating Google’s Reinforcement Learning for Chip Macro Placement

Igor L. Markov, igor.markov@gmail.com

## Abstract

Reinforcement learning (RL) for physical design of silicon chips in a Google 2021 Nature paper stirred controversy due to poorly documented claims that raised eyebrows and attracted critical media coverage. The Nature paper withheld most inputs needed to produce reported results and some critical steps in the methodology. But two separate evaluations filled in the gaps and demonstrated that Google RL lags behind human designers, behind a well-known algorithm (Simulated Annealing), and also behind generally-available commercial software. Crosschecked data indicate that the integrity of the Nature paper is substantially undermined owing to errors in the conduct, analysis and reporting.

**About the Author:** Dr. Igor L. Markov worked on Physical Design of Integrated Circuits for over 20 years [13, 15, 28, 33], coauthored dozens of highly-cited peer reviewed papers in the field, as well as a Physical Design textbook [43]. During his tenure at the University of Michigan, he supervised three doctoral dissertations on chip floorplanning and circuit placement. Dr. Markov coauthored algorithms and software packages for chip floorplanning and circuit placement that were commercialized and used to design commercial chips. He served as an editor of top EDA journals and of the two-volume Handbook of EDA [35]. He is an IEEE Fellow and an ACM Distinguished Scientist, as well a Nature author and reviewer. Dr. Markov previously worked on EDA at Synopsys, on Search at Google, and taught chip design at Stanford. He is a Research Scientist at Meta working on Machine Learning and a director at Nova Ukraine focusing on humanitarian relief.

The work reported here started when the author was affiliated with the University of Michigan, but opinions presented here are the author’s and do not represent his past, current or future employers.

For a successful technology, reality must take precedence over public relations, for Nature cannot be fooled.  
— Richard Feynman, the Challenger Accident Report

## 1 Introduction

The Nature paper [1] by Google researchers was published two years ago, broadly advertised as a Machine Learning breakthrough in chip design. It addressed a challenging problem to optimize locations of circuit components on a chip and described applications to five Google TPU chip blocks, implying that no better methods were available at the time in academia or industry. The paper generalized the claims beyond chip design to suggest that Reinforcement Learning (RL) outperforms state of the art in combinatorial optimization.

Surprisingly for such claims, [1] showed no results on public test examples (benchmarks [15]) and did not share the TPU chip blocks used or comparable examples. Source code, released 7 months after publication [3] to support [1] after initial controversy (as covered in [45, 46, 47, 50, 52]), was missing key parts necessary to reproduce the methods and results (as explained in [7, 55]). Over a dozen researchers [45, 46, 52, 55] from Google and academia questioned the claims of [1], ran extensive experiments, and raised further concerns [5, 7] about [1].

Confusingly, the then-head of Google Brain, Dr. Zoubin Ghahramani, a Google VP, [tweeted](#) on April 7, 2022 [44] “Google stands by this work published in Nature on ML for Chip Design, which has been independently replicated, open-sourced, and used in production at Google,” apparently referring to reproduction by another Google team (Sergio Guadarrama’s), and without specifying what aspects were reproduced. Google engineers updated the open-source release [3] many times since, filling in some missing pieces but not all [7]. A single chip-design example was added to [3] but wouldn’t have been sufficient even if it showed a clear win for

Google’s RL code (it didn’t [7]). Apparently, the only openly claimed independent (outside Google) reproduction of techniques in [1] was developed in Fall 2022 by UCSD researchers [7].<sup>1</sup> They had to reverse-engineer key components missing from [3] and completely reimplement the Simulated Annealing (SA) baseline [7] since none was provided in [3]. Google released no TPU chip design blocks used in [1] (nor sanitized equivalents), ruling out full external reproduction of results. So, the UCSD team shared [6] their experiments on modern, public chip designs: SA and commercial EDA tools beat Google RL code [3].

Given the earlier exodus of Machine Learning (ML) researchers from Google, reporters from the New York Times and Reuters covered this controversy in 2022 [45, 46] and discovered that well before the Nature paper was submitted its claims were disputed by Google researchers tasked with evaluating the work. The lead authors of the Nature paper [1] complained of persistent allegations of fraud [47], and the ML researcher who voiced those allegations was fired by Google. The dissenter sued Google for wrongful termination under California whistleblower-protection laws: February 2023 court documents [50], filed under penalty of perjury, reveal allegations of fraud and scientific misconduct related to the results in [1]. The first two authors of [1] left Google within months of controversial media coverage. A senior coauthor of [1] also left [48]. Media coverage in Spring 2023 touched on numerous aspects of the controversy, such as misrepresentation of results to potential cloud-services customers [52], reproducibility of results in the Nature paper [55], and the UCSD team’s attempts to settle the dispute via careful experiments [56].

In this note, Section 2 reviews the chip-design task solved in [1, 3] and technical background, it also introduces the secondary sources we use [4, 5, 6, 7]. Section 3 enumerates initial suspicions about [1]. Section 4 shows that many of them were confirmed. Section 5 checks if [1] improved SOTA. Section 6 outlines technical responses by authors of [1] to critiques. Section 7 speculates how the work from [1] may be used in practice. Section 8 draws conclusions about [1] and its methods, etc.

<sup>1</sup>Efforts by Prof. Andrew B. Kahng at UCSD were praised by Dr. Jeff Dean (the most senior author of the Nature paper [1] and then a Google SVP) in his recorded Dec 2, 2022 workshop keynote [49]. Additional disclosures: as UCSD efforts were starting, Prof. Kahng publicly stated [56] that he was Reviewer #3 of [1]. In the 1990s, Prof. Kahng supervised the doctoral dissertation of the author of

## 2 Background

The Nature paper [1] focuses on one step in the physical design for silicon chips [35, 43], where circuit components (small *gates* or *standard cells*, as well as memory arrays and reusable subcircuits) are represented by rectangles within the chip canvas as in Figure 1. Future wires connecting components are captured by the *circuit netlist* before wire routes are known: a netlist is an unordered set of *nets*, each naming components that should be connected. The length of a net depends on components’ locations and on wire routes; long routes are undesirable. The *macro placement* problem addressed in [1] seeks  $(x, y)$  locations for large circuit components (*macros*) so that (i) their rectangles do not overlap, (ii) the remaining components can be placed well to optimize chip layout [24, 33, 40].

**Circuit placement as an optimization task.** After  $(x, y)$  locations of all components are known, wires that connect components’ I/O pins are routed. Routes impact chip metrics (for power, timing/speed, etc). The optimization of  $(x, y)$  locations starts with simplified estimates of wirelength without wire routes. Pin locations  $(x_1, y_1)$  and  $(x_2, y_2)$  may be connected by horizontal and vertical wire segments in many ways, but the shortest route length is  $|x_1 - x_2| + |y_1 - y_2|$ . For multiple locations  $\{(x_i, y_i)\}_i$ , this estimate generalizes to

$$\text{HPWL} = (\max_i x_i - \min_i x_i) + (\max_i y_i - \min_i y_i) \quad (1)$$
 HPWL stands for *half-perimeter wirelength*, where the *perimeter* is taken of the *bounding box* of points  $\{(x_i, y_i)\}_i$  [24, 33, 43]. It is easy to compute and sum over many interconnects. This sum correlates with total routed wirelength reasonably well. HPWL is invariant under shifts of  $(x, y)$  locations. When  $(x, y)$  locations are scaled by a factor  $\gamma > 0$ , HPWL also scales by  $\gamma$ . This makes HPWL optimization scale-invariant and appropriate for all semiconductor technology nodes.<sup>2</sup> Algorithms that optimize HPWL typically extend to more precisely optimize routed wirelength and technology-dependent chip metrics, so HPWL optimization is a precursor [13, 18, 25, 29, 30, 31, 32, 33, 34, 36, 40]:

- used as an initial test for new algorithms and software, which are extended to handle more accurate objectives *after* they get close to state of the art (within 5% HPWL or so).

this note on large-scale VLSI placement at UCLA.

<sup>2</sup>With semiconductor technology scaling, sizes of macros may scale differently, but placement algorithms should handle a variety of macro sizes.

| Google Team 1<br>(Nature authors + coauthors)               | Google Team 2<br>+ external coauthors                    | UCSD Team                                                                                                                |
|-------------------------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| Circuit Training (CT) repo & FAQ [3]<br>ISPD 2022 paper [4] | Stronger Baselines [5]                                   | MacroPlacement repo & FAQ [6]<br>ISPD 2023 paper [7]                                                                     |
| 5 proprietary TPU blocks<br><br>ariane (public) [3]         | 20 proprietary TPU blocks<br>17 public IBM circuits [17] | 17 public IBM circuits [17]<br>2× ariane (public) [6, 7]<br>2× MemPool (public) [6, 7]<br>2× BlackParrot (public) [6, 7] |

Table 1: Secondary sources published by the teams and chip designs for which they report results. The IBM circuits [17] are ICCAD 2004 benchmarks. [7] built 3 designs with 2 semiconductor technologies.

- is followed by optimizations of sophisticated objectives that include HPWL (such as the proxy cost function used by RL in [1]).

Widely adopted algorithmic frameworks for placement do not use ML [24, 25, 33, 43, 40] and can be described as: (i) Simulated Annealing (SA), (ii) partitioning-driven, and (iii) analytical. SA, developed in the 1980s [8, 10] and dominant through mid-1990s [11], starts with an initial layout (e.g., random) and modifies it using a sequence of actions, such as component moves and swaps, of prescribed length. To ensure overall improvement, some actions may temporarily sacrifice quality. SA tends to produce great results on smaller layouts (up to 100K placeable components) but takes long runtimes for large layouts. Partitioning-driven methods [13, 19, 20, 22, 21] view the circuit connectivity (the netlist) as a hypergraph and use established software packages to subdivide it into partitions with stronger connectivity within the partitions (than across). These methods run faster than SA, capture global netlist structures, and were dominant for some 10 years. Yet, the mismatch between partitioning and placement objectives (Equation 1) leave room for improvement [21]. Analytical methods approximate Equation 1 by closed-form functions amenable to established optimization methods. *Force-directed placement* [9] from the 1980s models connections between circuit components by springs and finds component locations to balance out spring forces [43]. In the 2000s, advanced analytical placement techniques attained superiority [24, 33, 36, 40] on all large modern public benchmark sets, including those with macros and routing data [23, 26, 27, 29, 36]. RePlAce [36] from UCSD runs much faster than SA and partitioning-based methods, but loses in quality on some small netlists.

The Nature paper [1] focuses on large circuit components (*macros*) among numerous small com-

ponents. This *macro placement* problem was clearly formulated in the early 2000s [12, 14, 16], and is generally addressed within *mixed-size placement* [19, 21] where all circuit components are placed on a fixed-size canvas. In 2004, the author of this note and his then doctoral student Saurabh Adya released a suite of benchmarks [17] for evaluating mixed-size placement algorithms. Those benchmarks evaluate the HPWL objective (Equation 1) that remains appropriate for all semiconductor technology nodes. Hence, they saw significant use in the literature, e.g., [18, 19, 20, 21, 36].

**Key sources.** The Nature paper [1] solves mixed-size placement by first placing macros and then placing small components with commercial software. [1] places macros one at a time using a trained Reinforcement Learning (RL) policy and force-directed placement. It runs for a scheduled time and optimizes a proxy cost function that blends HPWL, component density, and routing congestion. [1] claims better results compared to baselines: (i) macro placement by human chip designers, (ii) Simulated Annealing, and (iii) the RePlAce software from UCSD, which uses no RL.

Among secondary sources discussed in the context of [1] (Table 1), we prefer scholarly papers [5, 4, 7] and draw on open-source repositories and FAQs as needed [3, 6]. We crosscheck claims from three nonoverlapping groups of researchers: those associated with [1], the Stronger Baselines paper [5] and the UCSD paper [7]. Consistent claims from different groups are even more trustworthy when backed by numerous benchmarks. Both Google Team 2 and the UCSD team included highly-cited experts on floorplanning and placement with extensive publication records and several key references cited in [1] (such as [27, 33, 36] and others), as well as experience developing academic and commercial floorplanning and placement tools beyond Google.

### 3 Initial doubts

While the Nature paper [1] was sophisticated and impressive, its research plan had notable shortfalls. For one, proposed reinforcement learning (RL) was presented as being capable of broader combinatorial optimization (a field that includes puzzle-like tasks such as the Traveling Salesman Problem, Vertex Cover, Bin Packing). But instead of illustrating this with canonical problem formulations and easy-to-configure test examples [39], it solved a specialty task (macro placement for chip design) and used proprietary Google TPU circuit design blocks, giving results on only 5 out of 20 available blocks. Further, the RL formulation did not directly handle chip metrics and optimized a simplified *proxy* function that included HPWL (see Section 2), but was not evaluated for pure HPWL optimization on open circuit benchmarks, as is routine in the literature [13, 15, 18, 19, 20, 21, 25, 24, 29, 33, 36, 40]. New ideas in placement are typically evaluated in research contests on industry chip-design examples released as public benchmarks [40, Section 6.1], e.g., [24, 30, 31, 32, 33, 34]. But [1] neglected contest benchmarks.

Some aspects of [1] looked suspicious, as it (1) did not substantiate several claims and withheld key aspects of experiments, (2) claimed improvements in noisy metrics that the proposed technique did not optimize, (3) relied on techniques with known handicaps that undermined performance in similar circumstances, and (4) may have misconfigured and underreported their baselines. We spell these out in Sections 3.1-3.4 — confirming even a fraction of specific concerns would put the top-line claims and conclusions of [1] in serious doubt.

#### 3.1 Unsubstantiated claims and ungenerous reporting

Several significant omissions can be understood by readers without background in chip design.

**U1.** With “fast chip design” in the title [1], the authors only described improvement in design-process time as “days or weeks” to “hours” without giving per-design time or breaking it down into stages. It was unclear if “days or weeks” for baseline design process included the time for functional design changes, idle time, inferior EDA tools, etc.

**U2.** The claim of RL’s six-hour runtimes per design [1, Abstract] excluded RL training on 20 TPU design blocks. [1] gave results on only 5 blocks, with training likely taking longer than actual use. Prior

methods did not need or use training, thus pausing the clock for training was misleading. Also, RL runtimes only cover macro placement, but RePlace and industry tools place all circuit components.

**U3.** [1] focused on placing macros, but did not give the number, sizes or shapes of macros in each TPU chip block used, nor area utilization, etc.

**U4.** [1] gives results on only five TPU blocks, with unclear statistical significance, but high-variance metrics produce noisy results (Table 2). Using more examples is common (Table 1).

**U5.** [1] was silent on the qualifications and the level of effort of the human chip designer(s) outperformed by RL. Reproducibility aside, those results could be easy to improve (as shown in [7] later).

**U6.** [1, Abstract] claimed improved “area”, but chip area and macro area did not change in [1], and standard-cell area did not change during placement (also see the 0.00 correlation in Table 2).

#### 3.2 A flawed optimization proxy

The chip design methodology in [1] uses physical synthesis to generate circuits for further layout optimization (physical design). The proposed RL technique places macros of those circuits to optimize a simplified *proxy cost function*. Then, a commercial EDA tool is invoked to place the remaining components (standard cells). The remaining operations (including power-grid design, clock-tree synthesis and timing closure [25, 43]) are outsourced to an unknown third party [1, 2]. Results are evaluated with respect to routed wirelength, area, power, and two circuit-timing metrics: TNS and WNS.<sup>3</sup>

Per [1], the proxy cost function did not perform circuit timing analysis [43] needed to evaluate TNS and WNS.<sup>4</sup> Therefore, it was misleading to claim in [1] that the proposed RL method led to TNS and WNS improvements on five TPU design blocks (without performing variance-based statistical significance tests). TNS and WNS were optimized at later steps unrelated to RL [1].

#### 3.3 Use of handicapped techniques

To researchers familiar with mixed-size placement, the proposed methodology *looked* handicapped in

<sup>3</sup>TNS = Total Negative Slack, WNS = Worst Negative Slack. These metrics measure violations of timing constraints (slack) by adding violations along all critical paths or using the worst violations. These metrics are noisy since chip timing is often determined by a handful of paths, and small changes to macro locations may change timing a lot.

<sup>4</sup>Proxy values correlate poorly with TNS and WNS [7].

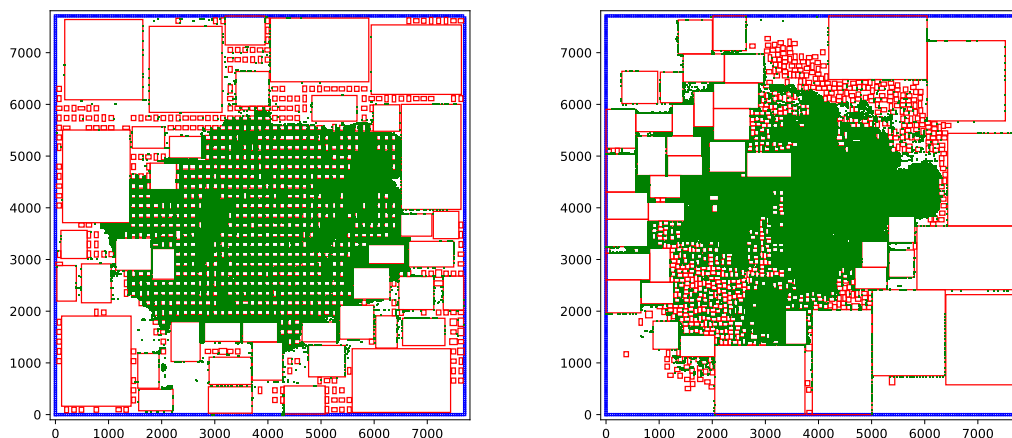


Figure 1: Layouts from [5, Figure 2] with macros in red and standard cells in green, locations produced by RL (left) and RePlace (right) for the **ibm10** benchmark from [17]. Limiting macro locations to a coarse grid (left) leads to spreading of small macros (red squares on a grid) and elongates connecting wires: from 27.5 (right) to 44.1 (left) for **ibm10** [5, Table 1]). High area utilization and many macros of different sizes make the ICCAD 2004 benchmarks [17] challenging compared to [1] and [2, page 43].

its reliance on multiple outdated techniques, making it difficult to improve State of the Art (SOTA).

**H1.** Proposed RL used exorbitant CPU/GPU resources compared to SOTA. Hence, claimed “fast chip design” (presumably, due to fewer unsuccessful design attempts) required careful substantiation.

**H2.** Placing macros one by one, known as a *constructive approach* to floorplanning [43], is one of the simplest approaches. Simulated Annealing can swap and shift macros, and make other incremental changes. Analytical methods simultaneously adjust locations of multiple components. Even when one-by-one placement was enhanced with Deep Reinforcement Learning, it looked handicapped.

**H3.** [1] used circuit-partitioning (clustering) methods similar to partitioning-based methods from 20 years ago [13, 20, 22, 21, 25, 43]. Those techniques are known to diverge from interconnect optimization objective [21, 43]. By placing macros using a clustered netlist *without gradual layout refinement*, RL might run into the same problem.

**H4.** [1] limited macro locations to a grid, whereas SOTA methods [36] do not impose such a constraint. We illustrate the difference in Figure 1. Even if RL can run without gridding, it might not scale to large enough circuits without gridding.

**H5.** The use of force-directed placement from the 1980s [9] in [1] left obvious room for improvement.<sup>5</sup>

<sup>5</sup>In [42], Google Team 1 uses a modern method (DREAM-Place [41] derived from RePlace) instead of force-directed placement but improves only proxy costs, not chip metrics.

### 3.4 Questionable baselines

The Nature paper [1] used several baselines to claim the superiority of proposed techniques. We already mentioned in Section 3.1 that the human baseline was undocumented and not reproducible.

**B1.** The main results of [1] are in [1, Table 1], where chip metrics are reported for five TPU design blocks. But comparisons to SA in [1] are not reported in terms of those chip metrics.

**B2.** [1] mentions that SA was used to postprocess the results of RL, but [1] does not present ablation studies to evaluate the impact of SA in terms of chip metrics.

**B3.** RePlace [36] was used as a baseline in [1] in a way inconsistent with its intended use. As Section 2 explains, analytical methods achieve superiority on circuits with millions of movable components, but RePlace was not intended for clustered netlists with a reduced number of components — it should be used directly without clustering (for details, see [36, 5, 7]). Clustering can handicap performance due to a mismatch between placement and partitioning objectives [21], and also by unnecessarily creating “chunky” clusters that are more difficult to place without overlaps.

**B4.** [1] did not describe how macro locations in SA were initialized, suggesting that [1] used a naive approach that could be improved.

Later, [5] identified more handicaps in the SA baseline in [1], and [7] confirmed them (Section 4).

## 4 Additional evidence

Months after the Nature publication [1], more data became available in [2, 3, 4, 5], followed by the first wave of controversial media coverage [45, 46, 47]. Nature editors released the peer review file for [1] including authors’ rebuttals: in the lengthy back-and-forth with reviewers [2] the authors assured reviewers that macro locations were not modified after placement by RL, confirming coarse-grid placement of macros. Among several contributions, [5] implemented the request of Nature Reviewer #3 [2] and benchmarked Google’s technique on 17 public chip-design examples [17]: prior methods decisively outperformed Google RL. Professors Patrick Madden (SUNY Binghamton) and Jens Lienig (TU Dresden) publicly expressed doubts about the Nature paper [45, 46]. As researchers noted gaps in the Google open-source release [3], such as the grouping (clustering) flow, Google engineers released more code (but not all), prompting more questions (see below). Another year passed, and [6, 7] expanded on the initial suspicions in several ways. They demonstrated that not limiting macro placement allows human designers and commercial EDA tools (separately) to outperform results produced by Google code [3]. [7, Table 2] estimated rank correlation of the proxy cost function optimized by RL to chip metrics used in [1, Table 1], and [7, Table 3] estimated the mean and standard deviation for chip metrics after RL-based optimization. We give a summary in Table 2, where rank correlations are low for all chip metrics, while TNS and WNS are noisy. Hence, the optimization of TNS and WNS in [1] relied on a flawed proxy and produced results in [1, Table 1] of dubious statistical significance. We note that  $\sigma/\mu > 0.5$  for TNS on Ariane-NG45 (also on BlackParrot-NG45 in [7, Table 3]). In the second round of critical media coverage, [1] was questioned by Profs. William Swartz (UT Dallas), Patrick Madden (SUNY Binghamton), and Moshe Vardi (Rice) [52, 55].

We list additional revelations and further questions.

**The human baseline in [1] wasn’t entirely human after all.** Per CT FAQ [3]: “our strongest baseline is the physical design team working directly with the assistance of commercial autoplacers, and we outperform this baseline (see “manual” baseline in Table 1 of our Nature article).” Since the methodology of this team is not explained, one is left with comparisons to fully human baselines and to fully automated commercial EDA tools, as is done in [7].

### Data leakage between training and test data?

Per [1], “as we expose the policy network to a greater variety of chip designs, it becomes less prone to overfitting.” But Google Team 1 showed later in [4, Figure 7] that pretraining on “diverse TPU blocks” did not improve quality of results. Pretraining on “previous netlist versions” improved quality somewhat. Pretraining RL and evaluating it on similar designs could be a serious flaw in methodology of [1].<sup>6</sup> As Google did not release TPU designs or per-design statistics, we cannot compare training and test data.

**Inconsistencies in claimed runtimes.** [1] claims six-hour runtimes, but public presentations by Google Team 1 annotate the blurred green-blue-white chip image in [1, Extended Data Figure 5] with runtimes ranging from 12 to 24 hours [37, 38] and identical total wirelength (55.42m).

### A middling Simulated Annealing baseline.

The Stronger Baselines paper [5] from Google Team 2 demonstrated that straightforward improvements (adding “move” and “shuffle” actions to “swap”, “shift” and “mirror” actions) to Simulated Annealing (SA) used by Google Team 1 in [1] helped SA produce better results than RL in a shorter amount of time when optimizing the same objective function. [7] reproduced the conclusions of [5] with an independent implementation of SA and found that SA results had less variance than RL results. Additionally, [5] proposed an improved macro-initialization heuristic for SA and equalized compute times when comparing RL to SA.

### Undisclosed use of $(x, y)$ locations from commercial tools.

Strong evidence and confirmation by Google engineers are mentioned in the UCSD paper [7] that [1] withheld a critical step of macro placement. When clustering the input netlist (CT merge [3]), it read in a placement to prevent clustering components with faraway locations. To produce  $(x, y)$  locations of macros, [1] used initial  $(x, y)$  locations of all circuit components produced by commercial EDA tools from Synopsys [3, Issue #25]. The lead authors of [1] confirmed using this step, claiming it was unimportant [51]. But it improved key metrics by 7-10% in [7]. Thus, the results in [1] needed algorithmic steps missing from [1].

**More undocumented techniques** were itemized in [7], which mentioned discrepancies between the Nature paper [1], their source code [3] and the ac-

<sup>6</sup>Such a methodology could help chip designers iterate on design changes faster, but that was not described in [1].

| Chip metrics $\rightarrow$        | area        | routed wirelength | power       | WNS         | TNS         |
|-----------------------------------|-------------|-------------------|-------------|-------------|-------------|
| Rank correlation to RL proxy cost | <b>0.00</b> | <b>0.28</b>       | <b>0.05</b> | <b>0.2</b>  | <b>0.05</b> |
| Mean $\mu$                        | 247.1K      | 834.8             | 4,978       | -100        | -65         |
| Standard deviation $\sigma$       | 1.652K      | 4.1               | 272         | 28          | 36.9        |
| $\sigma/ \mu $                    | 0.01        | 0.00              | 0.05        | <b>0.28</b> | <b>0.57</b> |

Table 2: Evaluating the soundness of the proxy cost used with RL in [1] and the noisiness of reported chip metrics after RL-based optimization. We summarize data from [7, Table 2] on the Kendall rank correlation of chip metrics to the RL proxy cost and from [7, Tables 3 and 4] on statistics for chip metrics (only Ariane-NG45 design data is shown, but data for BlackParrot-NG45 shows similar trends). As expected, purely-additive metrics (standard-cell area, routed wirelength and chip power) exhibit low variance, but the TNS and WNS metrics, that measure timing-constraint violations, have high variance.

tual code used for chip design at Google. These discrepancies included specific weights of terms in the proxy cost function, a different construction of the adjacency matrix from the circuit, and several “blackbox” elements of [3] available as binaries with no source code or full description in [1]. [5] and [6, 7] offer missing descriptions. Moreover, the results in [1] did not match the methods in [1] because key components were missing. And neither results nor methods were reproducible from descriptions.

**An inconclusive benchmark.** Google’s RL code [3] lost to prior methods on most chip-design examples in [5, 7] except for Ariane — the only example released in support of [1]. But when macros of Ariane were shuffled [7], chip metrics moved very little. Thus, Ariane was not a challenging benchmark.

**Likely limitations.** The Nature paper [1] did not disclose major limitations of its methods but promised success in broader combinatorial optimization. The Ariane design image in [1, Extended Data Figure 4] shows macro blocks of identical sizes: a potential limitation. Yet, [1] does not report basic statistics per TPU block: the number of macros and their shapes, design area utilization, and the fraction of area taken by macros. Based on [2, page 43] and the guidance from Google engineers to the authors of [7], it appears TPU blocks had area utilization on the order of 60%. Commercial chips are often denser, and can use macros of different sizes. Poor performance of Google RL on challenging public benchmarks from [17] used in [5, 7] (illustrated in Figure 1) may indicate undisclosed limitations. Another possible limitation is poor handling of preplaced (fixed) macros, common in industry layouts, but not discussed in [1]. By interfering with preplaced macros, gridding (see H4) can impact usability in practice. Poor performance on public benchmarks may also be due to overfitting to proprietary TPU designs.

## 5 Did [1] improve SOTA?

Empirical validation of research ideas typically seeks to improve State of the Art (SOTA) results. The claims of improving TPU designs in [1] would probably not make much sense if commercial EDA tools for chip placement were known to considerably outperform proposed methods. [1] claimed that humans produced better results than commercial EDA tools, but gave no substantiation. When license terms complicate publishing comparisons to commercial EDA tools,<sup>7</sup> one compares to academic software and to other prior methods, with the proviso that small improvements are not compelling. [5] and [6, 7] took different approaches to comparing methods from [1] to baselines, but cumulatively reported comparisons to commercial EDA tools, to human designers, to prior university software, and to two independent custom implementations of Simulated Annealing.

- [5] followed the descriptions in [1] and did not supply initial placement information. [6, 7] sought to replicate what Google actually used to produce results (without description in [1]).
- The authors of [5] from Google Team 2 had access to TPU design blocks and evaluated the impact of pretraining. It was small at best.<sup>8</sup>
- The authors of [6, 7] lacked access to Google training data and code, so they followed instructions by Google in [3] that explained how to obtain results similar to those in [1] without

<sup>7</sup>The lawsuit [50] alleges that better performance of commercial tools was known at Google before [1] was submitted.

<sup>8</sup>A consistent conclusion was reported in [4, Figure 7] by Google Team 1 — training on diverse designs does not help, and improvements are seen only when training in earlier versions of the same design. In December 2022 Dr. Jeff Dean, a then Google SVP and the most senior author of [1], also confirmed that RL did well without pretraining [49].

| ↓ Designs / Tools → | Google CT | Cadence CMP | UCSD SA |
|---------------------|-----------|-------------|---------|
| Ariane-NG45         | 32.31     | 0.05        | 12.50   |
| BlackParrot-NG45    | 50.51     | 0.33        | 12.50   |
| MemPool-NG45        | 81.23     | 1.27        | 12.50   |

Table 3: Runtimes in hours for three mixed-size placement tools and methodologies on three large chip modern designs reported in the arXiv version of [7]. Google CT - Circuit Training code supporting RL in the Nature paper, used without pretraining. Cadence CMP - Concurrent Macro Placer (commercial EDA tool). SA - Simulated Annealing implemented at UCSD following [5] given 12.5 hours of runtime in each case. CT and SA are used only to place macros, the remaining components are placed by a commercial EDA tool whose runtime is not included. Cadence CMP performs placement of all circuit components. By quality of results in [7] (not shown here), Cadence CMP leads, followed by Simulated Annealing, followed by Google CT. [6] additionally evaluated Cadence CMP versions by year and concluded that performance and runtime on these examples did not appreciably change between the versions since 2019.

pretraining. They also reimplemented SA following instructions in [5] and introduced several new chip-design examples (Table 1).

- [6, 7] but not [5] performed comparisons using chip metrics and using a commercial EDA tool (Cadence CMP), which outperformed Google RL. When running RePlace in this context, [7] used only macro locations produced by RePlace and placed standard cells with the same commercial software used after Google CT/RL [1, 3] (more details below).
- [6, 7] but not [5] repeated SA vs. RL comparisons for several configurations (those in [1], those in [3], and additional ones suggested by Google engineers). The results were consistent.
- [6, 7] demonstrated that a specific chip designer from IBM outperformed Google RL [3], whereas [5] did not use human baselines.

For comparisons that can be crosschecked, [5] and [6, 7] report qualitatively similar conclusions.

As pointed out in [5], RePlace was used in [1] in a way inconsistent with its intended use. As a mixed-size placer, RePlace expects a circuit netlist with macros and standard cells, as a large number of separate components. Instead, the comparisons in [1] suppressed the advantage of RePlace by clustering standard cells into a few large chunks.<sup>9</sup> With proper use of RePlace, [5] and, independently, [7] produce strong results for RePlace on well-known public ICCAD 2004 benchmarks.<sup>10</sup> In comparisons on recent designs [6, 7] in

terms of chip metrics, RePlace also shows stronger results, but still loses to Google CT/RL because its placements are harder to route (the losses are much smaller than those reported in [1]). Notably, [5, 6, 7] used RePlace in fast mode and not high-quality mode. Also, congestion mitigation was not used. To strengthen this RePlace baseline, one can use the congestion-driven mode in RePlace [36], and/or combine RePlace with routability-improvement techniques used in [1] (but not disclosed in [1], per [7]) such as cell bloating. Other techniques for routability improvement [18, 25, 33, 29] can be used too.

As explained in [5], the implementation of Simulated Annealing used in [1] was handicapped. Removing the handicaps (in the same source code base) improved results. When properly implemented, SA produces better solutions than Google CT/RL [3] faster, when both are given the same proxy cost function. This is shown consistently in [5, 7] on 17 widely used ICCAD 2004 benchmarks [17] and in [7] on several modern design benchmarks. Compared to Google CT/RL [3], SA consistently improves wirelength and power metrics. For circuit timing metrics TNS and WNS, it produces less noisy results that are within the RL noise margin [7]. Recall that the proxy function optimized by SA and RL does not include timing metrics [1].

Improving upon SOTA requires improving upon all prior baselines. Google CT/RL failed to improve by quality upon (i) human baselines, (ii) commercial EDA tools, and (iii) SA. It did not improve SOTA by runtime either (Table 3), and [1] did not disclose per-design data or design-process time. RePlace and SA gave stronger baselines than described in [1], when configured/implemented well.

<sup>9</sup>Per Section 2, analytical placers like RePlace [36] beat other methods on circuits with millions of components. With <100K components, earlier methods can be competitive.

<sup>10</sup>Comparing HPWL results to those in [19, 21], Google CT/RL [3] underperforms Feng Shui [20] circa 2005.



## 6 Responses to critiques of [1]

Despite critical media coverage [45, 46, 52, 56] and technical questions raised, especially about reproducibility [55], the authors of [1, 3] failed to remove remaining obstacles to reproducibility of the methods and results in [1]. The UCSD team put in significant engineering effort to overcome those obstacles, followed up on the work of Google Team 2 that criticized [1] in the internal paper [5], and, months later, elaborated on many of the critiques and doubts listed in Sections 3 and 4. Google Team 2 had access to Google TPU designs and the source code used in [1] before the CT GitHub repo [3] appeared. The UCSD authors of [6, 7] had access to CT [3] and benefited from a lengthy involvement of Google Team 1 engineers, but not access to SA code used in [1, 5] or several other key pieces of code missing from [3]. Nevertheless, the results in [5] and [6, 7] corroborate each other, and their qualitative conclusions are consistent. UCSD results for Ariane-NG45 closely match those by Google Team 1 engineers, and [7, Figure 4] shows that CT training curves of Ariane-NG45 generated at UCSD match those produced by Google Team 1 engineers. The paper [7] and the work were carefully reviewed by Google Team 1 engineers in Fall 2022 and Winter 2023, and no objections were raised [6, FAQ].

In March 2023, the two lead authors of [1], who left Google half a year earlier, objected to conclusions of [7] because the experiments in [7] lacked pretraining. As per MP FAQ in [6], pretraining wasn't used in [7] as Google's CT FAQ [3] stated that pretraining was not needed to reproduce results of [1], and because no pretraining data was released. Dr. Jeff Dean's presentation [49] mentioned strong RL results without pretraining. [5] evaluated pretraining using Google-internal code and did not see impact on qualitative conclusions.

Another objection [56] is that public circuit benchmarks [17] used in [5, 7] allegedly use outdated infrastructure. To the contrary, those benchmarks [17] have been evaluated with the HPWL objective, which scales accurately under geometric 2D scaling of chip designs and remains appropriate for all technology nodes (Section 2). Per [2], IC-CAD benchmarks were requested by Reviewer #3 of [1]. When [5, 7] implemented this ask, Google RL ran into trouble before routing became relevant: it lost by >20% in HPWL optimization (HPWL is the simplest yet important term of the proxy cost optimized by CT/RL [1, 3]).

The lead authors of [1] mention in [51] various

citations of [1] but so far have not listed successful reproductions of results from [1] that did not involve Google employees and cleared all known obstacles to reproduction. The studies in [5, 7] do not discuss other ways to use RL in chip design (than in [1, 3]), so we do not draw general conclusions.

## 7 Can the work in [1] be used?

The Nature paper [1] claimed applications to recent Google TPU chips, providing credence to the notion that those methods improved State of the Art. But aside from vague general claims, no chip-metric improvements were reported for specific production chips.<sup>11</sup> Section 5 shows that the methods of [1, 3] lag behind SOTA, e.g., Simulated Annealing from the 1980s [8, 10]. Moreover, a strong Google-internal implementation of SA from [5] could serve as a drop-in replacement of RL in [1, 3]. Without inside knowledge of details, we speculate how to reconcile the claimed use in TPUs with Google CT/RL lagging behind SOTA (per [5, 7]).

- Given the high variance of RL results in terms of chip-timing metrics TNS and WNS (due to low correlation with the proxy metric), trying many independent randomized attempts with variant proxy cost functions and hyperparameter settings may improve best-seen results [50], but at the cost of increasing runtime by many times over. But SA can also be used this way.
- Using in-house methods, even if somewhat inferior, is a common methodology in industry practice called *dogfooding* ("eat your own dog-food"). In a typical chip design, some blocks are not critical (do not affect chip speed) and are natural candidates for dogfooding.
- The Nature paper [1] mentioned that the results of RL were postprocessed by SA. The CT FAQ [3] explicitly disclaimed this postprocessing. Perhaps, postprocessing was used in the TPU design flow but not when comparing RL to SA. But since full-fledged SA consistently beats RL [5, 7], SA could substitute for RL (initial locations can be accommodated using an adaptive temperature schedule in SA).
- Google Team 1's follow-up [4] shows (in Figure 7) that pretraining improves results only

<sup>11</sup>[1, Table 1] shows results for TPU designs of an earlier generation, apparently produced *after* tape-out. Assuming substantial use in production, more recent TPU design blocks must have used [1, 3] for tape-out.

when pretraining on essentially the same design. Perhaps, Google is leveraging RL when performing multiple revisions to IC designs — a valid context, but it was not described in [1]. Moreover, commercial EDA tools are orders of magnitude faster when running from scratch [7], so the gap cannot be closed with RL pretraining and reward tuning from [1].

- Per [2, 7], TPU blocks exhibit much lower area utilization during placement (roughly 60%) than is common in modern chips. Configuring EDA tools for this context can be challenging. Court materials [50] indicate that misleading comparisons due to misconfigured EDA tools were flagged at Google but not corrected.

**Can Google CT code [3] be improved?** RL and SA are orders of magnitude slower than SOTA (Table 3), but pretraining (missing in CT) speeds up RL [4, Figure 7] by only several times.

The CT repository [3] now contains attempted improvements (such as upgrading [42] force-directed placement [9] to DREAMPlace [41]), but we have not seen serious improvements to chip metrics. Four major barriers to improving [1, 3] remain:

1. The proxy cost function optimized by RL does not reflect circuit timing [7], so improving RL may not help improving TNS and WNS.
2. [5, 7] show that SA outperforms RL when optimizing a given proxy function. Therefore, RL may lose even with a better proxy.
3. RL’s placing macros on a grid limits their locations (Figure 1). When a human designer was told not to force macros onto the grid, they found better locations [7]. Commercial EDA tools also avoid this limitation and (unsurprisingly) outperform Google CT/RL.
4. Clustering as a preprocessing step creates mismatches between placement and netlist partitioning objectives [21, 43].

## 8 Conclusions

This note focuses on the reproduction and evaluation of results in the Nature paper [1] from Google. Based on crosschecked newer data, we draw conclusions with ample redundancy (resistant to isolated mistakes): the integrity of the Nature paper [1] is substantially undermined owing to errors in the conduct, analysis and reporting of the study.

### 8.1 Conclusions about [1]

We crosscheck the results reported in [5, 6, 7] and also account for [2, 3, 4, 42], then summarize conclusions drawn from these works. This confirms many of the initial doubts about [1] and identifies additional deficiencies. As a result, it is clear that [1] is misleading in several ways, so the readers can have no confidence in the top-line claims of [1] and its conclusions. [1] did not improve SOTA while the methods and results of the original paper were not reproducible from the descriptions provided, contrary to stated editorial policies at Nature (see Section 8.3). The reliance on proprietary TPU designs for evaluation, along with ungenerous reporting of experiments, obstructed reproducibility of the methods and the results. These obstructions have not been removed, as of this writing. The authors of [5] had access to Google internal code whereas [7] reverse-engineered and/or reimplemented missing components. Google Team 2 and the UCSD team drew consistent conclusions from similar experiments, and each team made additional observations.

1. [1] reported improvements in several chip-timing metrics (TNS and WNS) that were not explicitly tracked or optimized by the proposed RL method, and those metrics did not correlate with the proxy objective used in optimization [7]. Those timing metrics were optimized in postprocessing by commercial EDA tools.
2. Design-process time improvements over human chip designers — a key claim of [1] — were not reported per design or detailed, and the humans involved were not documented. Later, it was clarified in the CT FAQ [3] that those human experts somehow used commercial auto-placement tools. However, [7] has shown how Google CT/RL was outperformed, in separate comparisons, by different human designers and by fully automated commercial EDA tools.
3. As first suggested in [5] and confirmed in [7], each algorithmic baseline described in [1] was lacking in some ways and not difficult to improve. As a result, prior methods outperform the methods of [1, 3] in quality and runtime.
4. The claim of six-hour runtimes for RL macro placement [1] is in doubt because the authors of [1] reported at conferences [38, 49] much longer runtimes next to the same chip im-

ages [1, Extended Data Figure 5] with identical total wirelength. Moreover, [6, 7] followed instructions from [3] (without pretraining) but observed much longer runtimes for Google code (Table 3). The Nature authors may have stopped the clock during pretraining, which took much longer than six hours. In any case, commercial tools run orders of magnitude faster (Table 3).

5. [1] withholds important details required to produce reported results. One of these details is the use of  $(x, y)$  locations produced by commercial software. Using these initial locations with Google’s RL technique markedly improves the  $(x, y)$  locations produced by it [7].
6. Improving the methods of [1] to make them competitive would be challenging due to the four barriers itemized in Section 7.

## 8.2 Conclusions for chip design

1. Machine-learning from entire chip designs is hard: learning from diverse designs might only improve runtime and not quality of results on any one design [4]. Learning from earlier versions of the same design can be useful in some cases, but should be compared to warm-starting Simulated Annealing with an initial placement and adaptive temperature schedule.
2. Griding and clustering methods (popular 20 years ago, but outperformed by “flat” methods) do not offer new capabilities at this point.
3. Using exorbitant CPU/GPU resources in [1] did not help outperform SOTA. It only complicated experimentation and reproducibility.
4. The work in [1, 3] made a keen observation that physical synthesis tools produce  $(x, y)$  locations usable as initial solutions for mixed-size placement. Sadly, this observation was not disclosed in the text of [1] but only used to improve results. As it is not specific to RL, it does not support RL [7], but initial placements were recently studied in [53] and can be reflected in future placement benchmarking efforts.
5. The modern open-source infrastructure [6] for evaluating macro placers developed for [7] can be used to check new ideas and software.<sup>12</sup>

<sup>12</sup>The design examples in [6] roughly match [1] in area utilization. Increasing area utilization would create harder

[6, 7] included in its evaluation a new ML-based macro placer AutoDMP [54] from Nvidia that produced promising results without using reinforcement learning. At the same time, older circuit benchmarks (such as [17]) remain relevant, difficult and practically useful, as they allow researchers to quickly estimate the direction of comparisons with minimal efforts for any semiconductor technology node.

## 8.3 Policy implications

**Google should follow Google AI principles** <https://ai.google/responsibility/principles>, in particular, Section “6. Uphold high standards of scientific excellence” that says:

“Technological innovation is rooted in the scientific method and a commitment to open inquiry, intellectual rigor, integrity, and collaboration... We aspire to high standards of scientific excellence...”

The April 7, 2022 tweet by ex-Head of Google Brain [44] appears to contradict the facts: the work in the Nature paper [1] was never fully open-sourced and was not independently reproducible because several key parts were not described in the paper or released in code. This was stated in [5] prior to the tweet, obvious from [3] (and publicly mentioned to the lead authors of [1] in March 2022), was later documented in detail in [6, 7] and explained in plain English in [55]. The still-underspecified use of [1] on Google TPU designs (only on selected blocks? trained and tested on similar blocks? postprocessed by Simulated Annealing?) does not counter strong evidence in [5, 6, 7] that [1] failed to improve SOTA. Many chips are designed every year without improving SOTA, whereas prior SOTA improvements in chip design did not merit Nature publications.

It remains unclear why Google did not allow publishing [5] (coauthored by the author of this note), especially after its results and conclusions were corroborated by the published paper [7] written at UCSD with lengthy involvement from Google. Granted, [5] and [6, 7] found major flaws in [1], but “a commitment to open inquiry, intellectual rigor, integrity, and collaboration” must protect legitimate research, even if it is politically inconvenient.

benchmarks, keeping in mind that higher area utilization decreases fabrication cost for mass-produced ICs.

**Nature Portfolio editorial policies should be followed broadly and rigorously.** Quoting from <https://www.nature.com/nature-portfolio/editorial-policies/reporting-standards>:

“An inherent principle of publication is that others should be able to replicate and build upon the authors’ published claims. A condition of publication in a Nature Portfolio journal is that authors are required to make materials, data, code, and associated protocols promptly available to readers without undue qualifications... After publication, readers who encounter refusal by the authors to comply with these policies should contact the chief editor of the journal.”

Clearly, when manuscript authors neglect requests for public benchmarking and obstruct reproducibility, their technical claims should be viewed with suspicion (especially if they later disagree with comparisons to their work [51]). In May 2022, [46] quoted a statement by Nature about [1]: “Issues relating to the paper have been brought to our attention and we are looking into them carefully”. In March 2023 [52] reported that

“Some academics have since urged Nature to review Google’s paper in light of UCSD’s study. In emails to the journal viewed by The Register, researchers highlighted concerns raised by Prof. Kahng and his colleagues, and questioned whether Google’s paper was misleading.”

Further, “Nature told The Register it is looking into Google’s paper... This process involves consultation with the authors and, where appropriate, seeking advice from peer reviewers and other external experts.” Soon after, [55] made a plain-language case that [1] lacked reproducibility.

---

As of this writing, Nature editors are investigating the Nature article [1], and the outcome of their investigation is unknown. Yet, it is in everyone’s interest to agree on clear and unequivocal conclusions about published scientific claims, free of misrepresentations. To this end, Nature editors and reviewers, as well as the authors and the research community, share the burden of responsibility.

**Acknowledgments.** This note would be impossible without the hard work and dedication to science of the authors of [5] and [6, 7].

## References

- [1] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan et al., “A Graph Placement Methodology for Fast Chip Design,” *Nature* 594 (2021), pp. 207-212. arXiv:2004.10746
- [2] Peer Review File for the Nature paper released by the editor: [https://static-content.springer.com/esm/art%3A10.1038%2Fs41586-021-03544-w/MediaObjects/41586\\_2021\\_3544\\_M0ESM1\\_ESM.pdf](https://static-content.springer.com/esm/art%3A10.1038%2Fs41586-021-03544-w/MediaObjects/41586_2021_3544_M0ESM1_ESM.pdf)
- [3] Circuit Training: An Open-source Framework for Generating Chip Floorplans with Distributed Deep Reinforcement Learning. [https://github.com/googleresearch/circuit\\_training](https://github.com/googleresearch/circuit_training),
- [4] Summer Yue, Ebrahim M. Songhori, Joe Wenjie Jiang, Toby Boyd, Anna Goldie, Azalia Mirhoseini, Sergio Guadarrama, “Scalability and Generalization of Circuit Training for Chip Floorplanning,” *ISPD 2022*: 65-70.
- [5] Sungmin Bae, Amir Yazdanbakhsh, Satrajit Chatterjee, Mingyu Woo, Igor. L. Markov, et al., “Stronger Baselines for Evaluating Deep Reinforcement Learning in Chip Placement”, March, 2022.<sup>13</sup> <https://statmodeling.stat.columbia.edu/wp-content/uploads/2022/05/MLcontra.pdf>
- [6] MacroPlacement Repo. <https://github.com/TILOS-AI-Institute/MacroPlacement>
- [7] Chung-Kuan Cheng, Andrew B. Kahng, Sayak Kundu, Yucheng Wang, Zhiang Wang, “Assessment of Reinforcement Learning for Macro Placement”, *ISPD 2023*, arXiv:2302.11014
- [8] Scott Kirkpatrick, C. Daniel Gelatt, Jr., Mario P. Vecchi, “Optimization by Simulated Annealing,” *Science* 1983, 220(4598), pp.671-680.
- [9] Chung-Kuan Cheng, Ernest S. Kuh, “Module Placement Based on Resistive Network Optimization,” *IEEE Trans. Comput. Aided Des.* 3(3), pp. 218-225 (1984)

---

<sup>13</sup>[50] disclosed some authors of the Stronger Baselines paper [5]. Apparently the authorship was well-known within Google from 2021.

- [10] Carl Sechen, Alberto Sangiovanni-Vincentelli, "The Timberwolf Placement and Routing Package," *IEEE Journal of Solid-State Circuits*, SC-20(2), pp. 510-522, Apr. 1985.
- [11] William Swartz, Carl Sechen, "Timing Driven Placement for Large Standard Cell Circuits," *DAC* 1995, pp. 211-215.
- [12] Andrew B. Kahng, "Classical floorplanning harmful?" *ISPD* 2000, pp. 207-213.
- [13] Andrew E. Caldwell, Andrew B. Kahng, Igor L. Markov, "Can recursive bisection alone produce routable placements?" *DAC* 2000, pp. 477-482.
- [14] Saurabh N. Adya, Igor L. Markov. "Fixed-outline floorplanning: enabling hierarchical design," *IEEE Trans. Very Large Scale Integr.* 11(6), pp. 1120-1135 (2003).
- [15] Richard Goering, "IC placement benchmarks needed, researchers say," *EE Times*, April 10, 2003.
- [16] Enno Wein and Jacques Benkoski, "Hard macros will revolutionize SoC design," *EE Times*, August 2004 <https://www.eetimes.com/hard-macros-will-revolutionize-soc-design/>
- [17] Saurabh Adya, Igor L. Markov, "IC-CAD04 Mixed-size Placement Benchmarks," <http://vlsicad.eecs.umich.edu/BK/ICCAD04bench/>
- [18] Chen Li, Min Xie, Cheng-Kok Koh, Jason Cong, Patrick H. Madden: Routability-driven placement and white space allocation. *ICCAD* 2004: 394-401
- [19] Ateen Khatkhate, Chen Li, Ameya R. Agnihotri, Mehmet Can Yildiz, Satoshi Ono, Cheng-Kok Koh, Patrick H. Madden, "Recursive bisection based mixed block placement," *ISPD* 2004, pp. 84-89.
- [20] Ameya R. Agnihotri, Satoshi Ono, Patrick H. Madden, "Recursive bisection placement: Feng Shui 5.0 implementation details," *ISPD* 2005, pp. 230-232.
- [21] Saurabh Adya, Igor L. Markov, "Combinatorial techniques for mixed-size placement," *ACM Trans. Design Autom. Electr. Syst.* 10(1): 58-90 (2005).
- [22] Jarrod A. Roy, Saurabh N. Adya, David A. Papa, Igor L. Markov: "Min-cut floor-placement," *IEEE Trans. Comput. Aided Des.* 25(7), pp. 1313-1326 (2006).
- [23] Jason Cong, Michalis Romesis, Joseph R. Shinnerl, Kenton Sze, Min Xie, "PEKO-MS benchmarks for mixed-size placement," 2006 [https://github.com/jshinnerl/pekoMS\\_2006\\_book](https://github.com/jshinnerl/pekoMS_2006_book)
- [24] Gi-Joon Nam, Jason Cong, "Modern Circuit Placement, Best Practices and Results," *Springer* 2007. Chapter 2 available as arXiv:2305.16413.
- [25] Charles J. Alpert, Dinesh P. Mehta, Sachin S. Sapatnekar, eds.: "Handbook of Algorithms for Physical Design Automation," *Auerbach*, 2008.
- [26] Jarrod A. Roy, Natarajan Viswanathan, Gi-Joon Nam, Charles J. Alpert, Igor L. Markov, "CRISP: Congestion reduction by iterated spreading during placement," *ICCAD* 2009: 357-362.
- [27] Myung-Chul Kim, Natarajan Viswanathan, Charles J. Alpert, Igor L. Markov, Shyam Ramji, "MAPLE: multilevel adaptive placement for mixed-size designs," *ISPD* 2012, pp. 193-200, 2011.
- [28] Myung-Chul Kim, Dongjin Lee, Igor L. Markov, "SimPL: an algorithm for placing VLSI circuits," *Commun. ACM* 56(6): 105-113 (2013)
- [29] Jason Cong, Guojie Luo, Kalliopi Tsota, Bingjun Xiao, "Optimizing routability in large-scale mixed-size placement," *ASP-DAC* 2013, pp. 441-446.
- [30] Myung-Chul Kim, Jin Hu, Natarajan Viswanathan: "ICCAD-2014 CAD contest in incremental timing-driven placement and benchmark suite," *ICCAD* 2014, pp. 361-366.
- [31] Myung-Chul Kim, Jin Hu, Jiajia Li, Natarajan Viswanathan, "ICCAD-2015 CAD Contest in Incremental Timing-driven Placement and Benchmark Suite," *ICCAD* 2015, pp. 921-926
- [32] Natarajan Viswanathan, Shih-Hsu Huang, Rung-Bin Lin, Myung-Chul Kim, "Overview of the 2015 CAD Contest at ICCAD," *ICCAD* 2015, pp. 910-911.

- [33] Igor L. Markov, Jin Hu, Myung-Chul Kim, “Progress and Challenges in VLSI Placement Research,” *Proc. IEEE* 103(11), pp. 1985-2003 (2015).
- [34] Shih-Hsu Huang, Rung-Bin Lin, Myung-Chul Kim, Shigetoshi Nakatake, “Overview of the 2016 CAD contest at ICCAD,” *ICCAD* 2016, pp. 38.
- [35] Luciano Lavagno, Igor L. Markov, Grant E. Martin, Louis K. Scheffer, eds.; “Electronic Design Automation for Integrated Circuits Handbook,” 2nd ed. (two volumes) *CRC*. 2016.
- [36] Chung-Kuan Cheng, Andrew B. Kahng, Ilgweon Kang, Lutong Wang; “RePlAce: Advancing Solution Quality and Routability Validation in Global Placement”, *IEEE TCAD* 38(9) (2018), pp. 1717-1730. <https://github.com/mgwoo/RePlAce>
- [37] Jeffrey Dean, keynote at ISSCC (February 18, 2020), reported at <https://www.nextplatform.com/2020/02/20/google-teaches-ai-to-play-the-game-of-chip-design/>
- [38] Young-Joon Lee, presentation at the CHIPS Alliance Deep Dive Cafe Talks, November 2020. [http://youtu.be/EKjlr2k\\_wBM](http://youtu.be/EKjlr2k_wBM).
- [39] Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, Evgeny Burnaev, “Reinforcement Learning for Combinatorial Optimization: A Survey,” *Computers and Operations Research* 134, October 2021, 105400; arXiv:2003.03600
- [40] Andrew B. Kahng, “Advancing Placement,” *ISPD* 2021, pp. 15-22.
- [41] Yibo Lin, Zixuan Jiang, Jiaqi Gu, Wuxi Li, Shounak Dhar, Haoxing Ren, Brucek Khailany, David Z. Pan. “DREAMPlace: Deep Learning Toolkit-Enabled GPU Acceleration for Modern VLSI Placement,” *IEEE Trans. Comput. Aided Des.* 40(4), pp. 748-761 (2021).
- [42] Zixuan Jiang, Ebrahim Songhori, Shen Wang, Anna Goldie, Azalia Mirhoseini, Joe Jiang, Young-Joon Lee, David Z. Pan, “Delving into Macro Placement with Reinforcement Learning,” *MLCAD*, 2021. arXiv:2109.02587
- [43] Andrew B. Kahng, Jens Lienig, Igor L. Markov, Jin Hu, “VLSI Physical Design: from Graph Partitioning to Timing Closure”, *Springer*, 2nd ed, 2022.
- [44] Zoubin Ghahramani, “Google stands by this work published in Nature”, *Twitter* April 7, 2022, <https://twitter.com/ZoubinGhahrama1/status/1512203509646741507?t=9kyvWT0gCXQ5coWCG7LvsA>
- [45] Daisuke Wakabayashi and Cade Metz, “Another Firing Among Google’s A.I. Brain Trust, and More Discord,” *The New York Times*, May 2, 2022. <https://www.nytimes.com/2022/05/02/technology/google-fires-ai-researchers.html>
- [46] Paresh Dave, “Google faces internal battle over research on AI to speed chip design,” May 3, 2022. <https://www.reuters.com/technology/google-faces-internal-battle-over-research-ai-speed-chip-design-2022-05-03/>
- [47] Tom Simonite, “Tension Inside Google Over a Fired AI Researcher’s Conduct,” *Wired*, May 31, 2022. <https://www.wired.com/story/google-brain-ai-researcher-fired-tension/>
- [48] “Lightmatter Names Top Google TPU Engineer Richard Ho as VP of Hardware Engineering,” *Business Wire*, August 2021 <https://www.businesswire.com/news/home/20220822005051/en/Lightmatter-Names-Top-Google-TPU-Engineer-Richard-Ho-as-VP-of-Hardware-Engineering>
- [49] Jeffrey Dean, “ML for Computer Systems at Google,” keynote at the SysML workshop at NeurIPS, Dec 2, 2022 <https://slideslive.com/38994456/machine-learning-for-computer-systems-at-google>
- [50] Lawrence A. Bohm, “Satrajit Chatterjee vs. Google”, first amended complaint, filed in the Superior Court of the Santa Clara County, February 2023. [https://regmedia.co.uk/2023/03/26/satrajit\\_vs\\_google.pdf](https://regmedia.co.uk/2023/03/26/satrajit_vs_google.pdf)
- [51] Anna Goldie, Azalia Mirhoseini, “Statement on Reinforcement Learning for Chip Design,” March 24, 2023 [https://drive.google.com/file/d/1jWUw6rUDcc7fuHu\\_iGeVDUkBXNJhHdd/view](https://drive.google.com/file/d/1jWUw6rUDcc7fuHu_iGeVDUkBXNJhHdd/view)

- [52] Katyanna Quach, “Google’s claims of super-human AI chip layout back under the microscope,” March 27, 2023. [https://www.theregister.com/2023/03/27/google\\_ai\\_chip\\_paper\\_nature/](https://www.theregister.com/2023/03/27/google_ai_chip_paper_nature/)
- [53] Pengwen Chen, Chung-Kuan Cheng, Albert Chern, Chester Holtz, Aoxi Li, Yucheng Wang, “Placement Initialization via Sequential Subspace Optimization with Sphere Constraints,” *ISPD 2023*, pp. 133-140.
- [54] Anthony Agnesina, Puranjay Rajvanshi, Tian Yang, Geraldo Pradipta, Austin Jiao, Ben Keller, Brucek Khailany, Haoxing Ren, “AutoDMP: Automated DREAMPlace-based Macro Placement,” *ISPD 2023*: 149-157.
- [55] Gregory Goth, “More Details, but Not Enough”, *Communications of the ACM*, 29 Mar 2023 <https://cacm.acm.org/news/271439-more-details-but-not-enough/fulltext>
- [56] Samuel Moore, “Ending an Ugly Chapter in Chip Design Study tries to settle a bitter disagreement over Google’s chip design AI”, *IEEE Spectrum*, 4 Apr 2023. <https://spectrum.ieee.org/chip-design-controversy>